

Aplicações Backend

Aula 3 – Tipos de dados estruturados - Vetores e Matrizes

Prof. Wanderson Santos

wanderson.santos@projecao.br



Roteiro da Aula



- Tipos de dados estruturados – Vetores
- Tipos de dados estruturados - Matrizes
- Exercícios.





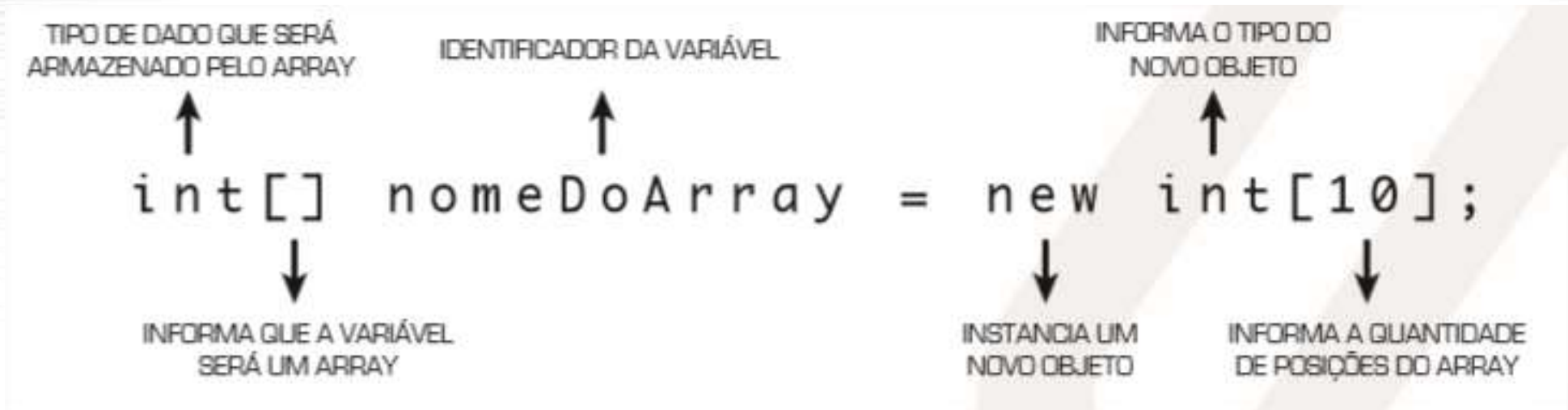
Vetores



- Quando desejamos armazenar uma grande quantidade de valores de um determinado tipo, podemos utilizar **arrays**.
 - Um **array** é um objeto que pode armazenar muitos valores de um determinado tipo (**unidimensional**).
-

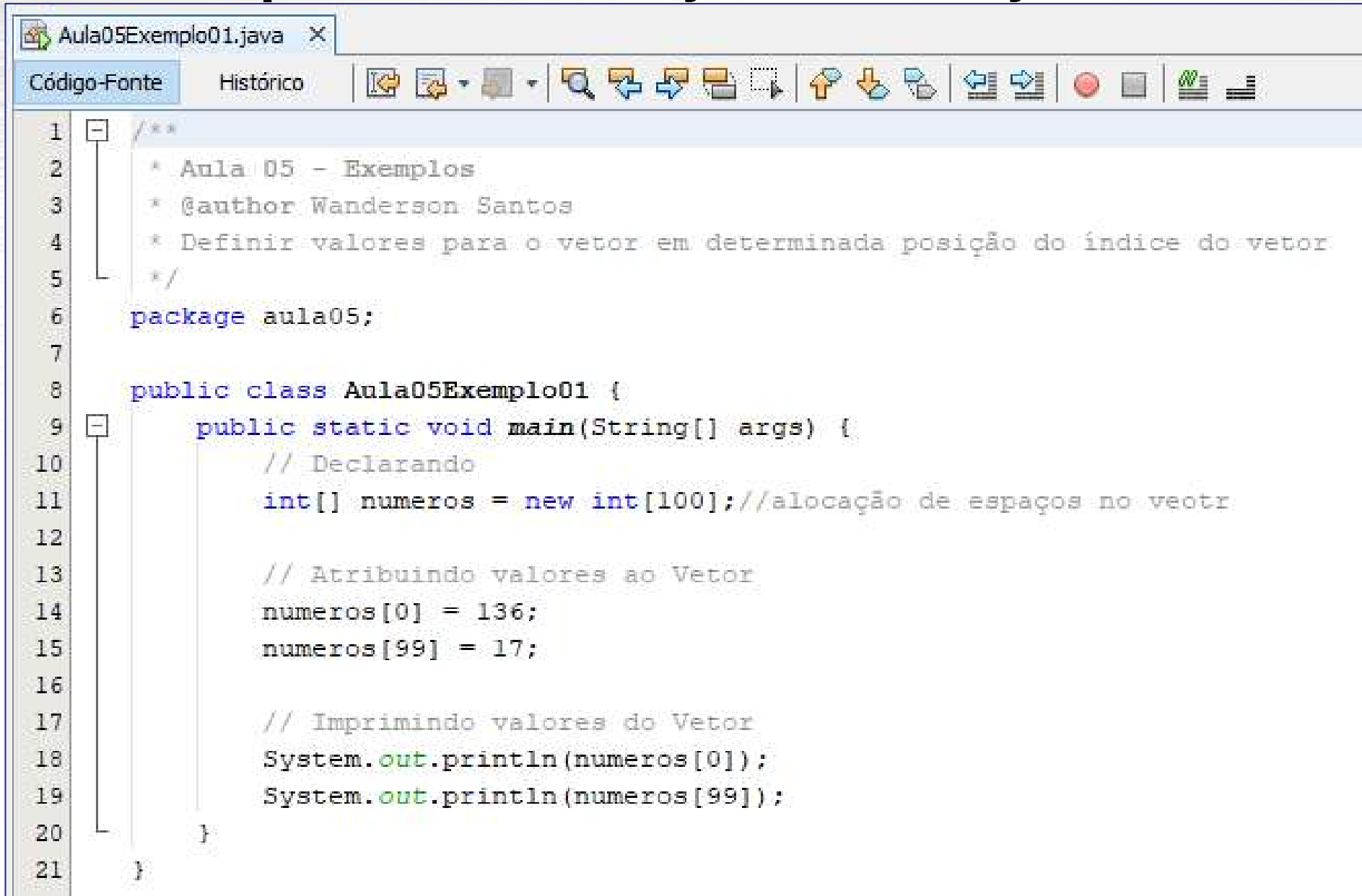
Vetores

- Como se declarar um array ou vetor no Java?



Vetores

- **Exemplo1** da declaração e atribuição direta:

A screenshot of a Java IDE window titled 'Aula05Exemplo01.java'. The window has a menu bar with 'Código-Fonte' and 'Histórico', and a toolbar with various editing icons. The code is displayed in a text area with line numbers 1 through 21 on the left. The code defines a package 'aula05', a public class 'Aula05Exemplo01', and a public static void method 'main'. Inside 'main', an integer array 'numeros' of size 100 is declared and allocated. Then, the first element 'numeros[0]' is assigned the value 136, and the last element 'numeros[99]' is assigned the value 17. Finally, the values at these two indices are printed to the console using 'System.out.println'.

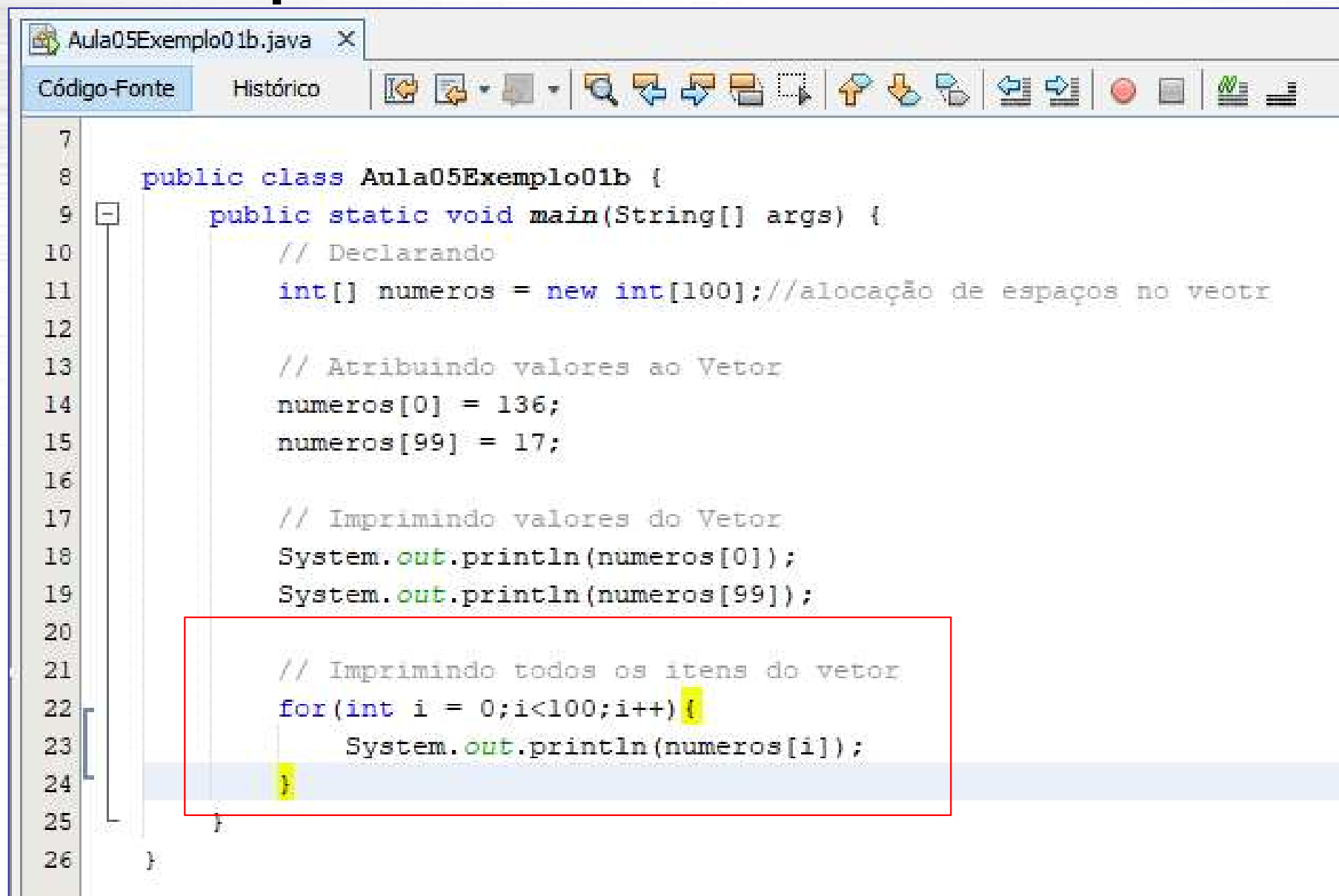
```
1  /**
2   * Aula 05 - Exemplos
3   * @author Wanderson Santos
4   * Definir valores para o vetor em determinada posição do índice do vetor
5   */
6  package aula05;
7
8  public class Aula05Exemplo01 {
9      public static void main(String[] args) {
10         // Declarando
11         int[] numeros = new int[100]; // alocação de espaços no vetor
12
13         // Atribuindo valores ao Vetor
14         numeros[0] = 136;
15         numeros[99] = 17;
16
17         // Imprimindo valores do Vetor
18         System.out.println(numeros[0]);
19         System.out.println(numeros[99]);
20     }
21 }
```

Vetores

- **Obs:** quando um **array** é criado com o comando **new**, todas as posições são inicializadas com os valores padrão (números são inicializados com 0, booleanos com false e referências com null).
-

Vetores

- Exemplo 01b

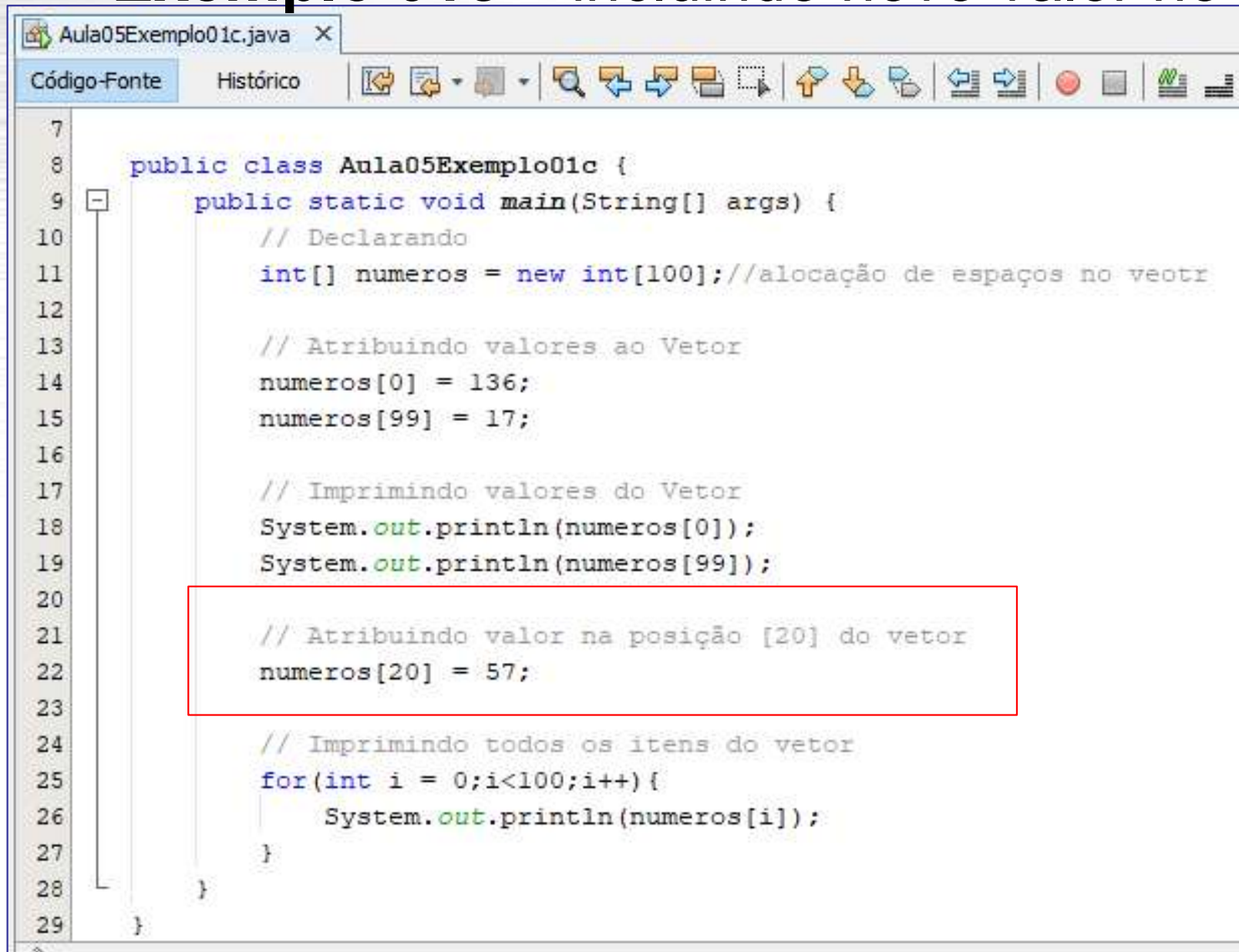
A screenshot of a Java IDE window titled 'Aula05Exemplo01b.java'. The window has a menu bar with 'Código-Fonte' and 'Histórico', and a toolbar with various icons. The code is as follows:

```
7  
8 public class Aula05Exemplo01b {  
9     public static void main(String[] args) {  
10         // Declarando  
11         int[] numeros = new int[100]; // alocação de espaços no vetor  
12  
13         // Atribuindo valores ao Vetor  
14         numeros[0] = 136;  
15         numeros[99] = 17;  
16  
17         // Imprimindo valores do Vetor  
18         System.out.println(numeros[0]);  
19         System.out.println(numeros[99]);  
20  
21         // Imprimindo todos os itens do vetor  
22         for(int i = 0; i < 100; i++) {  
23             System.out.println(numeros[i]);  
24         }  
25     }  
26 }
```

The code is displayed with line numbers on the left. A red rectangular box highlights the for loop starting at line 22 and ending at line 24. The IDE interface includes a toolbar with icons for file operations, navigation, and execution.

Vetores

- **Exemplo 01c** – incluindo novo valor no vetor



```
7
8 public class Aula05Exemplo01c {
9     public static void main(String[] args) {
10         // Declarando
11         int[] numeros = new int[100]; // alocação de espaços no vetor
12
13         // Atribuindo valores ao Vetor
14         numeros[0] = 136;
15         numeros[99] = 17;
16
17         // Imprimindo valores do Vetor
18         System.out.println(numeros[0]);
19         System.out.println(numeros[99]);
20
21         // Atribuindo valor na posição [20] do vetor
22         numeros[20] = 57;
23
24         // Imprimindo todos os itens do vetor
25         for(int i = 0; i < 100; i++) {
26             System.out.println(numeros[i]);
27         }
28     }
29 }
```


Vetores

- Definindo o tamanho do vetor dinamicamente a partir de uma variável (Exemplo 02);

```
6 package aula05;
7
8 public class Aula05Exemplo02 {
9     public static void main(String[] args) {
10         // Declarando Variaveis
11         int x = 10; // tamanho do vetor
12         int[] numeros = new int[x]; // alocação de espaços para o vetor
13         int i; // índice do vetor (ou 'posição' do vetor)
14
15         // Processando os 'n' elementos no vetor 'numeros'
16         for (i=0; i < x; i++){
17             numeros[i] = i; // armazenando valores no vetor
18         }
19
20         // Imprimindo todos os itens do vetor
21         for(i = 0; i < x; i++){
22             System.out.println(numeros[i]);
23         }
24     }
```

Vetores

- Exemplo 02: representação interna

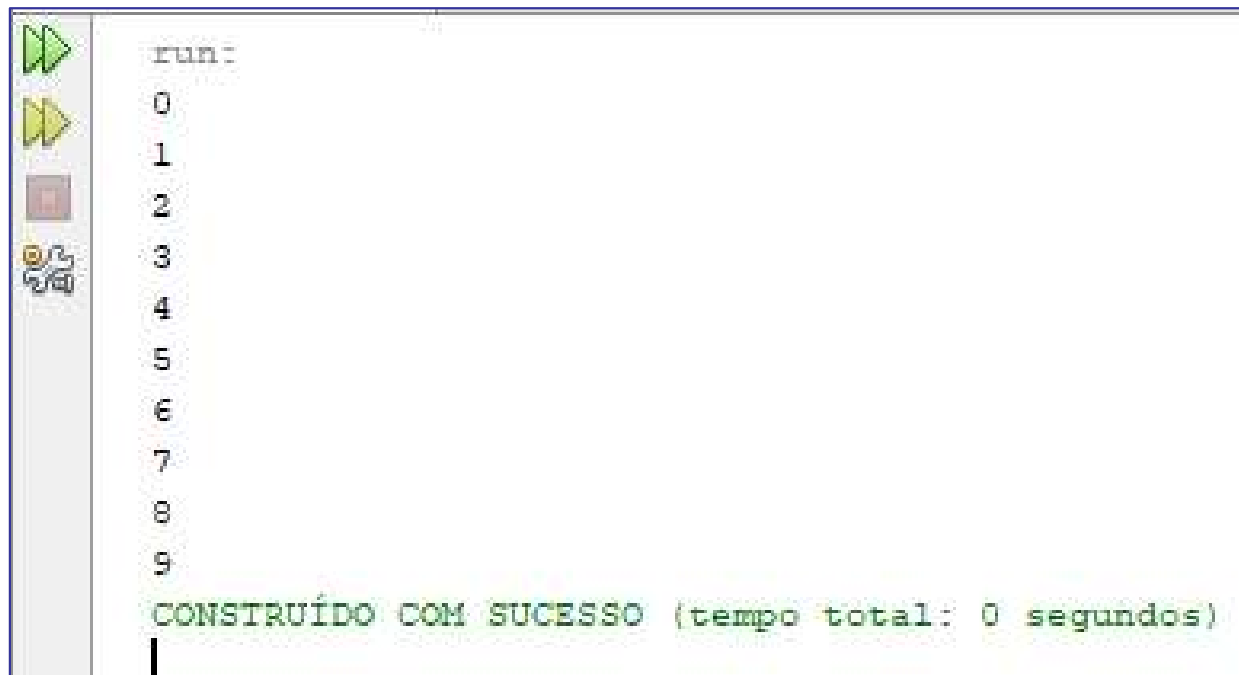
v[0]	v[1]	v[2]	v[3]	v[4]	v[5]	v[6]	v[7]	v[8]	v[9]
0	1	2	3	4	5	6	7	8	9

Vetores

- Exemplo 02: representação interna

v[0]	v[1]	v[2]	v[3]	v[4]	v[5]	v[6]	v[7]	v[8]	v[9]
0	1	2	3	4	5	6	7	8	9

- Saída:



```
run:
0
1
2
3
4
5
6
7
8
9
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Vetores

- **Acessando o conteúdo de um array:**
- Para acessar o conteúdo de um array, devemos escolher uma ou mais posições e utilizar a sintaxe abaixo:

```
int[] numeros = {100,87};  
System.out.println(numeros[0]);  
System.out.println(numeros[1]);
```

- Obs: Acessar posições fora do intervalo de índices de um array gera erro de execução. Ocorrerá a exception:
- **ArrayIndexOutOfBoundsException.**

Vetores

- **Percorrendo um Array:**
- Para percorrermos um array, utilizaremos a instrução de repetição for:

```
int[] numeros = new int[100];  
for(int i = 0; i < 100; i++) {  
    numeros[i] = i;  
}
```

Vetores

- Para percorrer um array, é necessário saber a quantidade de posições do mesmo. Essa quantidade é definida quando o array é criado através do comando **new**.
 - Nem sempre essa informação está explícita no código.
 - Por exemplo, considere um método que imprima na saída padrão os valores armazenados em um array. Provavelmente, esse método receberá como parâmetro um array e a quantidade de posições desse array não estará explícita no código fonte.
-

Vetores

- Vejamos este exemplo de Método que deve imprimir o conteúdo de um array de int:

```
void imprimeArray(int[] numeros) {  
    // implementação  
}
```

- Podemos recuperar a quantidade de posições de um array acessando o seu atributo **length**.

```
void imprimeArray(int[] numeros) {  
    for(int i = 0; i < numeros.length; i++) {  
        System.out.println(numeros[i]);  
    }  
}
```

Vetores

- Para acessar todos os elementos de um array, é possível aplicar o comando for com uma sintaxe um pouco diferente (**for each**).

```
void imprimeArray(int[] numeros) {  
    for(int numero : numeros) {  
        System.out.println(numero);  
    }  
}
```

Vetores

- **Operações**
- Nas bibliotecas da plataforma Java, existem métodos que realizam algumas tarefas úteis relacionadas a arrays. Veremos esses métodos a seguir.
- **Ordenando um Array:**

```
Arrays.sort();
```

Vetores

- Considere um array de String criado para armazenar nomes de pessoas. Podemos ordenar esses nomes através do método **Arrays.sort()**.

```
String[] nomes = new String[]{"rafael cosentino", "jonas hirata", "marcelo martins"}
Arrays.sort(nomes);

for(String nome : nomes) {
    System.out.println(nome);
}
```

Vetores

- Preenchendo um Array

`Arrays.fill()`.

```
int[] numeros = new int[10];  
java.util.Arrays.fill(numeros, 5);
```

Vetores

- Também podemos definir os valores de cada posição de um array no momento da sua criação utilizando as sintaxes abaixo:

```
int[] numeros = new int[2]{100,87};
```

```
int[] numeros = new int[]{100,87};
```

```
int[] numeros = {100,87};
```
